

server_refresh.ps1

Native PowerShell-uppdatering direkt på SQL Server

Datum: 2026-06-25

Bakgrund och syfte

Det befintliga uppdateringsskriptet (`morning_refresh.ps1`) körs idag på en operatörsdator och kräver att Git och Python är installerade. Om uppdateringen ska köras direkt på `INTSQLSERVER01` faller detta bort — Git och Python är inte serverprogram och bör inte installeras i en produktionsmiljö.

Lösningen är ett nytt PowerShell-script som gör exakt samma sak men utan externa beroenden. Allt skriptet behöver finns redan på varje Windows Server — inbyggda PowerShell-cmdlets och .NET-bibliotek som ingår i operativsystemet.

Resultatet är ett enda `.ps1`-script som kopieras till servern, registreras med Task Scheduler och sedan sköter sig självt.

1

Kontrollera databasanslutningen

Precis som idag testas anslutningen till `InternalStatistics` på `INTSQLSERVER01` innan något arbete påbörjas. Om databasen inte svarar loggas en varning och skriptet avslutar — nästa schemalagda körning försöker igen automatiskt. Ingen data går förlorad.

2

Hämta nya snapshot-filer från GitHub

Istället för `git pull` anropar skriptet GitHub:s REST API direkt med en läsbehörig nyckel (PAT). Det frågar vilka JSON-filer som finns i `raw/freshdesk/` och `raw/linear/`, jämför listan mot vad som redan är inläst i databasens `import_log`, och laddar bara ned de filer som ännu inte är inlästa. En typisk körning laddar ned 1–2 filer på några sekunder.

3

Läs in Freshdesk-data i bronze-lagret

Varje nedladdad Freshdesk-fil innehåller en lista med support-tickets i JSON-format. Skriptet tolkar JSON nativt i PowerShell och infogar bara tickets som är nya eller uppdaterade sedan förra körningen — samma inkrementella logik som idag. Filnamnet registreras i `import_log` så att samma fil aldrig laddas in två gånger.

4

Läs in Linear-data i bronze-lagret

Samma process för Linear-issues. JSON-strukturen är mer komplex med nästlade objekt för tillstånd, tilldelad person, projekt, team och etiketter — skriptet plattar ut dessa och lagrar allt i rätt kolumner. Etiketter sammanfogas till en pipe-separerad sträng, precis som idag.

5

Bygg om silver-lagret

När bronze-lagret är uppdaterat kör skriptet de två silver-SQL-skripten — ett för Freshdesk och ett för Linear. Skripten hämtas direkt från GitHub vid varje körning och körs via inbyggda .NET-bibliotek (`System.Data.SqlClient`). Det innebär att skriptet alltid använder den senaste versionen av silver-logiken utan att något behöver kopieras till servern manuellt. Silver-lagret byggs om från grunden varje gång — samma design som idag.

Krav på servermiljön

Innan skriptet installeras behöver IT bekräfta följande:

- ✓ **Utgående HTTPS (port 443) tillåten** från `INTSQLSERVER01` till `api.github.com` — skriptet hämtar data och SQL-skript härifrån
- ✓ **En mapp på servern** dit skriptet och loggfiler skrivs, t.ex. `D:\Scripts\opex-refresh\` — tjänstkontot behöver läs- och skrivbehörighet
- ✓ **GitHub PAT** — samma läsbehöriga nyckel som redan används; förvaras i `github_token.txt` i mappen, aldrig i själva skriptet
- ✓ **Databasbehörighet** — tjänstkontot ges `db_datareader` + `db_datawriter` + `EXECUTE` på `InternalStatistics`

Inga extra programvaror. Skriptet använder `System.Data.SqlClient` — ett .NET-bibliotek som ingår i Windows. Varken Python, Git eller ODBC-drivrutiner behöver installeras.

Installation

- 1 Skapa mappen på servern, t.ex. `D:\Scripts\opex-refresh\`
- 2 Kopiera skriptet (`server_refresh.ps1`) till mappen
- 3 Lägg filen `github_token.txt` i samma mapp
- 4 Öppna PowerShell **som Administratör** på servern och kör:

```
powershell -ExecutionPolicy Bypass -File "D:\Scripts\opex-refresh\server_refresh.ps1" -Register
```

Detta registrerar två Task Scheduler-uppgifter: en som kör refreshen varje timme från kl. 06:00, och en watchdog-uppgift som kontrollerar kl. 16:00 om refreshen lyckats.

- 5 Testa direkt: högerklicka på uppgiften `ServerRefresh_InternalStatistics` i Task Scheduler → **Kör**, kontrollera att `logs\server_refresh.log` avslutas med `=== Klar ===`
- 6 **Valfritt:** Skapa filen `teams_webhook_url.txt` i mappen med en Teams Incoming Webhook URL — watchdog-uppgiften skickar då automatiskt ett meddelande i Teams kl. 16:00 vid utebliven refresh

Autentisering mot SQL Server

Eftersom skriptet körs på samma maskin som SQL Server används **Windows Authentication** — tjänstkontot som kör Task Scheduler-uppgiften loggar in i databasen automatiskt utan lösenord i skriptet. IT behöver bara ge det kontot rätt behörighet på `InternalStatistics`.

Inga klartext-lösenord för databasen — varken i skriptet eller i en konfigurationsfil. Enda hemligheten som hanteras är GitHub PAT-nyckeln, som bara behövs för att läsa snapshot-filerna.

Om något går fel

SITUATION	VAD HÄNDER
Databasen svarar inte	Loggar varning, avslutar — försöker igen nästa timme
GitHub API svarar inte	Loggar fel, avbryter utan att ändra befintlig data
En JSON-fil är korrupt	Den filen hoppas över och loggas — övriga filer laddas normalt
Ingen lyckad körning kl. 16:00	Watchdog-uppgiften skriver en felpost till Windows Application Event Log (händelse-ID 1001, källa <code>OPEX-Refresh</code>) och skickar valfritt Teams-meddelande
GitHub PAT har gått ut	Loggar tydligt felmeddelande — ny nyckel får skapas och läggas i <code>github_token.txt</code>